



Minicurso de C

Aula 05: Funções e Ponteiros

1

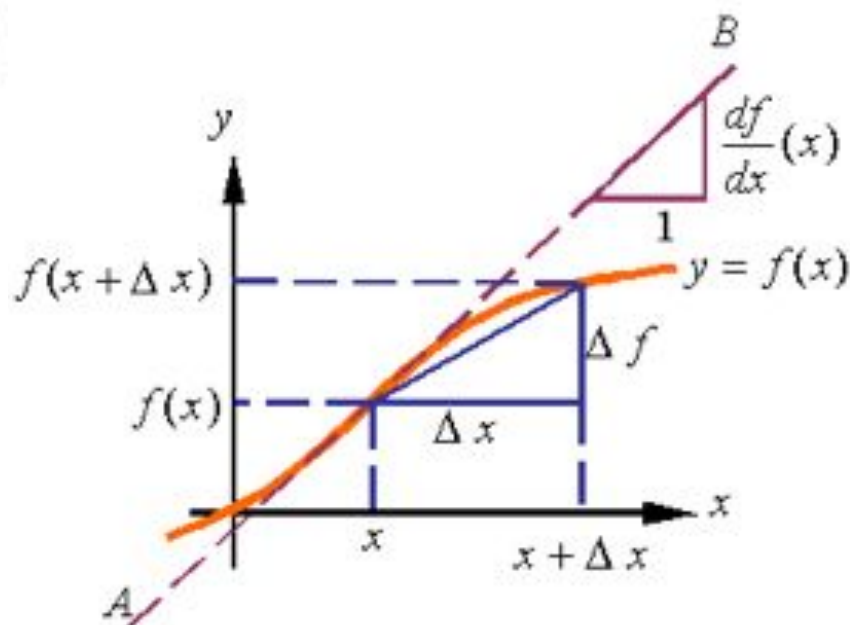
Funções



O que é uma função?



$$\begin{aligned}f'(3) &= \lim_{h \rightarrow 0} \frac{(3+h)^2 - 9}{h} \\&= \lim_{h \rightarrow 0} \frac{9 + 6h + h^2 - 9}{h} \\&= \lim_{h \rightarrow 0} \frac{6h + h^2}{h} \\&= \lim_{h \rightarrow 0} \frac{h(6 + h)}{h} \\&= \lim_{h \rightarrow 0} (6 + h) \\&= 6\end{aligned}$$





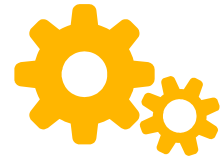
Uma função é uma **subrotina** que é definida e pode ser chamada a qualquer momento no código, podendo receber **parâmetros** e **retornar** ou não variáveis.

```
{  
    instruções;  
    retorno_da_função;  
}
```



Tipos de funções

- Funções que retornam valores: **int()**, **float()**, **double()**, **char()**
- Função que não retorna valor: **void()**
- Função principal: **int main()**



Exemplo de aplicação

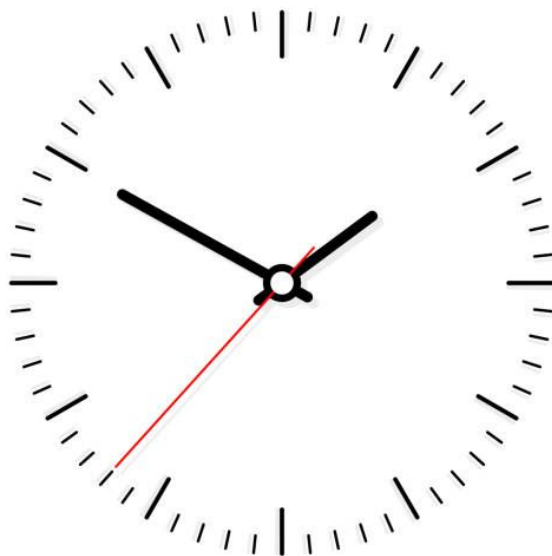
[ACESSE AQUI](#)

2

Ponteiros



O que são ponteiros?





- **Ponteiros** são variáveis que armazenam o **endereço de memória** de outras variáveis;
- Dizemos que um ponteiro “aponta” para uma variável quando contém o endereço da mesma;
- Os ponteiros podem apontar para qualquer tipo de variável. Portanto, temos ponteiros para int, float, double, etc;



Por que utilizar ponteiros?

- Ponteiros são muito úteis quando uma variável tem que ser acessada em diferentes partes de um programa;
- O código pode ter vários ponteiros espalhados por diversas partes do programa, “apontando” para a variável que contém o dado desejado;
- Caso esse dado seja alterado, não há problema algum, pois todas as partes do programa tem um ponteiro que aponta para o endereço onde reside o dado atualizado.



Aplicações dos ponteiros

- Passagem de parâmetros por referência;
- Manipulação de arrays;
- Alocação dinâmica de memória;
- Referência para listas, pilhas, árvores e grafos.

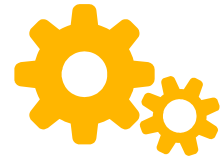


Declarando ponteiros

tipo *nome_ponteiro;

- **tipo**: é o tipo de dado da variável cujo endereço o ponteiro armazena;
- **nome_ponteiro**: O nome da variável ponteiro.
- O asterisco * neste tipo de declaração determina que a variável será um ponteiro;

Exemplo: **int *ptr;**



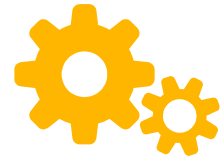
Exemplo de aplicação

[ACESSE AQUI](#)



Passagem de parâmetros

- **Passagem por valor:** permite usar dentro de uma função uma cópia do valor de uma variável, porém não permite alterar o valor da variável original (somente a cópia pode ser alterada);
- **Passagem por referência:** é passada para a função uma referência da variável, sendo possível alterar o conteúdo da variável original usando essa referência.



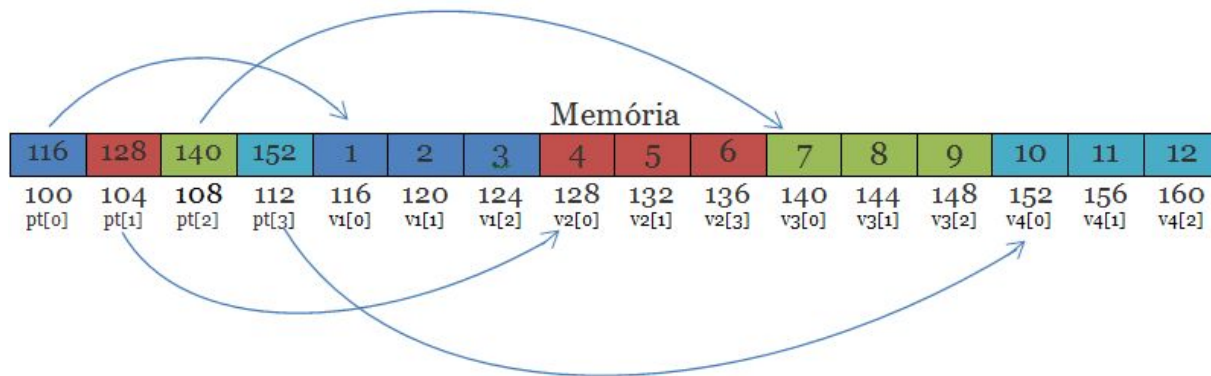
Exemplo de aplicação

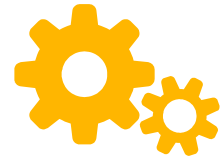
[ACESSE AQUI](#)



Array de ponteiros

```
1 int * pt [4]; //vetor de ponteiros do tipo inteiro
2 int v1[3] = {1, 2, 3}; //vetor 1 com três elementos
3 int v2[3] = {4, 5, 6}; //vetor 2 com três elementos
4 int v3[3] = {7, 8, 9}; //vetor 3 com três elementos
5 int v4[3] = {10, 11, 12}; //vetor 4 com três elementos
6
7 pt[0] = v1; //atribui o endereço do vetor1 para o ponteiro pt[0]
8 pt[1] = v2; //atribui o endereço do vetor2 para o ponteiro pt[1]
9 pt[2] = v3; //atribui o endereço do vetor3 para o ponteiro pt[2]
10 pt[3] = v4; //atribui o endereço do vetor4 para o ponteiro pt[3]
```





Exemplo de aplicação

[ACESSE AQUI](#)



Obrigado!

Alguma dúvida?